Scenario

Server Agent and Other Client Agents (2 or more)
No GUI only Console Outs
Run different agents using code
There will be two types Server and Client
Clients
need to discover the Service offered by the Server agent
Connect to server, ask for a service
This is not MAS only one Server AGENT
Example Classes
Java
Data structures:- Map, stack, …
Agent communication:- messages
JADE Agent Behaviors :
PLEASE DO NOT USE AI AGENTS

Please check the example agent classes

Should know how to compile and run

Example :- TicketBooking System

Create Folder(TicketBooking)-
| -> jade.jar
| -> TicketServerAgent.java
|-> TicketClientAgent.java

TicketServerAgent.java

```
import jade.core.Agent;
import jade.core.behaviours.CyclicBehaviour;
import jade.lang.acl.ACLMessage;
import jade.lang.acl.MessageTemplate;
import jade.domain.DFService;
import jade.domain.FIPAAgentManagement.DFAgentDescription;
import jade.domain.FIPAAgentManagement.ServiceDescription;
import jade.domain.FIPAException;
import java.util.*;

public class TicketServerAgent extends Agent {
    private Map<String, Integer> availableTickets;
    private Stack<String> bookingHistory;
```

```
    protected void setup() {
        System.out.println("Server Agent " + getAID().getName() + " is
ready.");

        // Initialize ticket inventory
        availableTickets = new HashMap<>();
        availableTickets.put("Movie-A", 50);
        availableTickets.put("Movie-B", 30);
        availableTickets.put("Concert-X", 100);
        availableTickets.put("Concert-Y", 75);

        bookingHistory = new Stack<>();

        // Register service in Yellow Pages (DF)
        DFAgentDescription dfd = new DFAgentDescription();
        dfd.setName(getAID());
        ServiceDescription sd = new ServiceDescription();
        sd.setType("ticket-booking");
        sd.setName("Ticket-Booking-Service");
        dfd.addServices(sd);

        try {
            DFService.register(this, dfd);
            System.out.println("Ticket Booking Service registered in DF");
        } catch (FIPAException fe) {
            fe.printStackTrace();
        }

        // Add behavior to handle requests
        addBehaviour(new TicketRequestHandler());
    }

    protected void takeDown() {
        try {
            DFService.deregister(this);
            System.out.println("Server Agent " + getAID().getName() + "
terminating.");
        } catch (FIPAException fe) {
            fe.printStackTrace();
        }
    }

    private class TicketRequestHandler extends CyclicBehaviour {
        public void action() {
            MessageTemplate mt =
MessageTemplate.MatchPerformative(ACLMessage.REQUEST);
            ACLMessage msg = myAgent.receive(mt);
```

```
            if (msg != null) {
                String content = msg.getContent();
                System.out.println("\n[SERVER] Received request: " +
content + " from " + msg.getSender().getLocalName());

                ACLMessage reply = msg.createReply();

                if (content.startsWith("LIST")) {
                    // Handle list request
                    StringBuilder response = new StringBuilder("Available
Tickets:\n");
                    for (Map.Entry<String, Integer> entry :
availableTickets.entrySet()) {
                        response.append(entry.getKey()).append(":
").append(entry.getValue()).append(" tickets\n");
                    }
                    reply.setPerformative(ACLMessage.INFORM);
                    reply.setContent(response.toString());
                    System.out.println("[SERVER] Sending ticket list");

                } else if (content.startsWith("BOOK")) {
                    // Handle booking request: BOOK|Movie-A|2
                    String[] parts = content.split("\\|");
                    if (parts.length == 3) {
                        String event = parts[1];
                        int quantity = Integer.parseInt(parts[2]);

                        if (availableTickets.containsKey(event)) {
                            int available = availableTickets.get(event);
                            if (available >= quantity) {
                                availableTickets.put(event, available -
quantity);
                                String booking =
msg.getSender().getLocalName() + "|" + event + "|" + quantity;
                                bookingHistory.push(booking);
                                reply.setPerformative(ACLMessage.AGREE);
                                reply.setContent("SUCCESS: Booked " +
quantity + " ticket(s) for " + event);
                                System.out.println("[SERVER] Booking
successful: " + event + " x" + quantity);
                            } else {
                                reply.setPerformative(ACLMessage.REFUSE);
                                reply.setContent("FAILED: Only " +
available + " tickets available for " + event);
                                System.out.println("[SERVER] Booking
failed: Insufficient tickets");
```

```
                        }
                    } else {
                        reply.setPerformative(ACLMessage.REFUSE);
                        reply.setContent("FAILED: Event " + event + "
not found");
                        System.out.println("[SERVER] Booking failed:
Event not found");
                    }
                } else {
                    reply.setPerformative(ACLMessage.NOT_UNDERSTOOD);
                    reply.setContent("Invalid booking format. Use:
BOOK|EventName|Quantity");
                }

            } else if (content.startsWith("HISTORY")) {
                // Handle history request
                StringBuilder response = new StringBuilder("Booking
History:\n");
                if (bookingHistory.isEmpty()) {
                    response.append("No bookings yet\n");
                } else {
                    Stack<String> temp = new Stack<>();
                    while (!bookingHistory.isEmpty()) {
                        String record = bookingHistory.pop();
                        response.append(record).append("\n");
                        temp.push(record);
                    }
                    while (!temp.isEmpty()) {
                        bookingHistory.push(temp.pop());
                    }
                }
                reply.setPerformative(ACLMessage.INFORM);
                reply.setContent(response.toString());
                System.out.println("[SERVER] Sending booking
history");

            } else {
                reply.setPerformative(ACLMessage.NOT_UNDERSTOOD);
                reply.setContent("Unknown command. Use: LIST,
BOOK|EventName|Quantity, or HISTORY");
            }

            send(reply);
        } else {
            block();
        }
    }
```

```
    }
}
```

TicketClientAgent.java

```
import jade.core.Agent;
import jade.core.AID;
import jade.core.behaviours.*;
import jade.lang.acl.ACLMessage;
import jade.lang.acl.MessageTemplate;
import jade.domain.DFService;
import jade.domain.FIPAAgentManagement.DFAgentDescription;
import jade.domain.FIPAAgentManagement.ServiceDescription;
import jade.domain.FIPAException;

public class TicketClientAgent extends Agent {
    private AID serverAgent;
    private int requestCount = 0;

    protected void setup() {
        System.out.println("Client Agent " + getAID().getName() + " is
ready.");

        // Add behavior to discover server
        addBehaviour(new OneShotBehaviour() {
            public void action() {
                DFAgentDescription template = new DFAgentDescription();
                ServiceDescription sd = new ServiceDescription();
                sd.setType("ticket-booking");
                template.addServices(sd);

                try {
                    System.out.println("[" + getLocalName() + "] Searching
for ticket-booking service...");
                    DFAgentDescription[] result =
DFService.search(myAgent, template);

                    if (result.length > 0) {
                        serverAgent = result[0].getName();
                        System.out.println("[" + getLocalName() + "] Found
server: " + serverAgent.getLocalName());

                        // Start interaction with server
                        addBehaviour(new TicketRequestBehaviour());
                    } else {
                        System.out.println("[" + getLocalName() + "] No
ticket-booking service found!");
                        doDelete();
```

```
                    }
                } catch (FIPAException fe) {
                    fe.printStackTrace();
                }
            }
        });
    }

    protected void takeDown() {
        System.out.println("Client Agent " + getAID().getName() + "
terminating.");
    }

    private class TicketRequestBehaviour extends SequentialBehaviour {
        public TicketRequestBehaviour() {
            // Step 1: Request ticket list
            addSubBehaviour(new RequestPerformer("LIST"));
            addSubBehaviour(new WaiterBehaviour(myAgent, 2000)); // Wait 2
seconds

            // Step 2: Try to book tickets
            String[] events = {"Movie-A", "Concert-X", "Movie-B"};
            int eventIndex = (int)(Math.random() * events.length);
            int quantity = (int)(Math.random() * 5) + 1;

            addSubBehaviour(new RequestPerformer("BOOK|" +
events[eventIndex] + "|" + quantity));
            addSubBehaviour(new WaiterBehaviour(myAgent, 2000));

            // Step 3: Request history (optional)
            if (Math.random() > 0.5) {
                addSubBehaviour(new RequestPerformer("HISTORY"));
                addSubBehaviour(new WaiterBehaviour(myAgent, 2000));
            }

            // Step 4: Try another booking
            eventIndex = (int)(Math.random() * events.length);
            quantity = (int)(Math.random() * 3) + 1;
            addSubBehaviour(new RequestPerformer("BOOK|" +
events[eventIndex] + "|" + quantity));
        }
    }

    private class RequestPerformer extends OneShotBehaviour {
        private String request;

        public RequestPerformer(String request) {
```

```
            this.request = request;
        }

        public void action() {
            ACLMessage msg = new ACLMessage(ACLMessage.REQUEST);
            msg.addReceiver(serverAgent);
            msg.setContent(request);
            msg.setConversationId("ticket-booking-" + requestCount++);

            System.out.println("\n[" + myAgent.getLocalName() + "]
Sending: " + request);
            myAgent.send(msg);

            // Add behavior to receive response
            myAgent.addBehaviour(new
ResponseReceiver(msg.getConversationId()));
        }
    }

    private class ResponseReceiver extends CyclicBehaviour {
        private String conversationId;
        private boolean received = false;

        public ResponseReceiver(String convId) {
            this.conversationId = convId;
        }

        public void action() {
            if (!received) {
                ACLMessage reply = myAgent.receive();
                if (reply != null &&
reply.getConversationId().equals(conversationId)) {
                    received = true;
                    System.out.println("[" + myAgent.getLocalName() + "]
Received response:");
                    System.out.println(reply.getContent());

System.out.println("----------------------------------------");
                } else {
                    block();
                }
            }
        }

        public boolean done() {
            return received;
        }
```

```
    }
}

# Compile
javac -cp jade.jar TicketServerAgent.java javac -cp jade.jar
TicketClientAgent.java
# Run (Console only)
 java -cp .:jade.jar jade.Boot server:TicketServerAgent
client1:TicketClientAgent client2:TicketClientAgent
```

JADE Ticket Booking System - Compilation and Execution Guide

Prerequisites

- Java JDK 8 or higher installed
- JADE library (jade.jar) downloaded from https://jade.tilab.com/

Project Structure

```
project/
├── TicketServerAgent.java
├── TicketClientAgent.java
└── jade.jar
```

Step 1: Compile the Agents

Windows

```
javac -cp jade.jar TicketServerAgent.java
javac -cp jade.jar TicketClientAgent.java
```

Linux/Mac

```
javac -cp jade.jar TicketServerAgent.java
javac -cp jade.jar TicketClientAgent.java
```

Step 2: Run the System

Option A: Run with GUI (Recommended for learning)

Windows:

```
java -cp .;jade.jar jade.Boot -gui server:TicketServerAgent
client1:TicketClientAgent client2:TicketClientAgent
```

Linux/Mac:

```
java -cp .:jade.jar jade.Boot -gui server:TicketServerAgent
client1:TicketClientAgent client2:TicketClientAgent
```

Option B: Run Console Only (No GUI)

Windows:

```
java -cp .;jade.jar jade.Boot server:TicketServerAgent
client1:TicketClientAgent client2:TicketClientAgent
client3:TicketClientAgent
```

Linux/Mac:

```
java -cp .:jade.jar jade.Boot server:TicketServerAgent
client1:TicketClientAgent client2:TicketClientAgent
client3:TicketClientAgent
```

Understanding the Command

```
java -cp .;jade.jar jade.Boot [options] agent1:Class agent2:Class ...
```

- **`-cp .;jade.jar`** - Classpath includes current directory and jade.jar
- **`jade.Boot`** - JADE bootstrap class
- **`-gui`** - Optional: Shows JADE GUI
- **`server:TicketServerAgent`** - Creates agent named "server" of class TicketServerAgent
- **`client1:TicketClientAgent`** - Creates agent named "client1" of class TicketClientAgent

Expected Console Output

```
Server Agent server@... is ready.
Ticket Booking Service registered in DF
Client Agent client1@... is ready.
[client1] Searching for ticket-booking service...
[client1] Found server: server

[client1] Sending: LIST
[SERVER] Received request: LIST from client1
[SERVER] Sending ticket list
[client1] Received response:
Available Tickets:
```

```
Movie-A: 50 tickets
Movie-B: 30 tickets
Concert-X: 100 tickets
Concert-Y: 75 tickets
----------------------------------------

[client1] Sending: BOOK|Concert-X|3
[SERVER] Received request: BOOK|Concert-X|3 from client1
[SERVER] Booking successful: Concert-X x3
[client1] Received response:
SUCCESS: Booked 3 ticket(s) for Concert-X
----------------------------------------
```

System Features

Server Agent Features

1. Service Registration - Registers in JADE Directory Facilitator (Yellow Pages)
2. Data Structures Used:
    - `Map<String, Integer>` - Stores event names and available tickets
    - `Stack<String>` - Maintains booking history (LIFO)
3. Handles Multiple Request Types:
    - `LIST` - Returns all available tickets
    - `BOOK|EventName|Quantity` - Books tickets
    - `HISTORY` - Shows booking history

Client Agent Features

1. Service Discovery - Uses DF to find ticket-booking service
2. JADE Behaviors Used:
    - `OneShotBehaviour` - For single actions (discovery, sending requests)
    - `CyclicBehaviour` - For continuous listening to responses
    - `SequentialBehaviour` - For ordered execution of tasks
    - `WaiterBehaviour` - For adding delays between actions
3. Agent Communication:
    - Uses ACL Messages (REQUEST, INFORM, AGREE, REFUSE)
    - Conversation IDs for tracking message threads

Adding More Clients

Simply add more clients to the command:

```
java -cp .:jade.jar jade.Boot server:TicketServerAgent
client1:TicketClientAgent client2:TicketClientAgent
client3:TicketClientAgent client4:TicketClientAgent
```

Each client will independently discover the server and make random booking requests.

Troubleshooting

Error: ClassNotFoundException

- Ensure jade.jar is in the same directory
- Check classpath syntax (; for Windows, : for Linux/Mac)

Error: No service found

- Make sure server agent starts before clients
- Check DF registration was successful

Agents don't communicate

- Verify all agents are on same JADE platform
- Check console for error messages

Key Concepts Demonstrated

1. No GUI - Pure console output showing agent interactions
2. Service Discovery - Clients find server via DF
3. Agent Communication - REQUEST/RESPONSE message pattern
4. Data Structures - Map for inventory, Stack for history
5. JADE Behaviors - Multiple behavior types for different tasks
6. NOT MAS - Single server agent, multiple clients (client-server model)
7. Concurrent Requests - Multiple clients can interact simultaneously